`ComplexGitSync` 7.3.0– Python API

Nicolas Flipo

October 27, 2026

Contents

1 Direct Python Object API **2**

2 Programmatic Project Configuration **2**

3 From Empty Registry to READY via `.gts` **2**

4 Object-Level Tag Propagation **3**

5 The Operating Space: one name, one id, one Merkle root **3**

6 READY Methods in the Client API **4**

7 Landing a Branch: `merge` and `refresh_private` **7**

8 Targeting a Single Path: `add` and `remove` **8**

9 `.gitignore` Lifecycle Sync and Commit Identity **9**

10 Forcing a Clone Protocol **9**

11 Repository Discovery: `discover_repos` **10**

12 Submodule Migration: `import_submodules` **10**

13 Adopting a Submodule Tree: `init_from_submodules` **11**

14 Looking at a memory: `memory_status`, `memory_list`, `memory_show`, `memory_explore` **12**
- 14.1 Starting a memory's history over: `memory_reboot` 14
- 14.2 Keeping a branch's memory when its project branch is merged: `memory_merge` . 14
- 14.3 Recording one piece of agent work: `self_history_add` 14
- 14.4 Adopting self-history: `self_history_adopt` 15
- 14.5 Reading self-history: `memory_self_history` 16

15 Machine-readable output: `status_json` and `verify_json` **16**

16 Environment observation and drift **17**

17 A memory as a repository: `memory_init`, `memory_clone`, `memory_push` **17**
- 17.1 A developer's tree with no memory: `memory_setup` 18

18 The shared register: `omniscience_mount`, `omniscience_record`, `omniscience_register` and `omniscience_observe` **18**

19 Asking the register: `omniscience_get` and its eight siblings **19**

20 One writer per workspace: the workspace lock **20**

21 Ledger Integrity: `verify` **21**

22 Diagnosing and Repairing a Git Situation: `autofix` **21**

1 Direct Python Object API

This chapter shows how to manipulate `ComplexGitSync` objects directly (without the CLI wrapper), including loading a `.gts` snapshot, reconstructing the runtime registry, mirroring identity into `GitTree`/`GitRepo`, and propagating a tag target.

For normal Multi-repo Sync usage, prefer the CLI lifecycle documented in the README and Getting Started guide. The reference test topology is CGSil1 (`https://gitlab.com/CGS_test/CGSil1`): `initialise -> pull -> checkout/add/commit/push -> freeze_release -> list_releases -> load_release`.

The lower-level object/API lifecycle remains: `load(.cgs) -> .gts LOADED -> expand(.cgs|.gts) -> .gts PENDING -> validate(.cgs|.gts) -> .gts READY -> pull(.cgs|.gts) -> checkout(.gts) -> add -> git(tree,"commit",msg) -> git(tree,"push") -> git(tree,"tag",name) -> freeze_relea`

2 Programmatic Project Configuration

Project configuration does not require the CLI or an existing `.cgs` file. The public facade accepts authoring values non-interactively and delegates normalization, static validation, and optional serialization to `cgs_format.py`. It performs no Git or network operations.

```
from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
document = client.configure(
    "GX4G",
    ["codeberg:GX4G/GX4G"],
    output_path="GX4G.cgs",  # optional
)
reference_tree = document.to_git_tree()
```

`configure` is the `.cgs` writer and has no CLI command of its own. `discover -write`, `submodules init`, and direct `initialise -project/-repo` authoring are the adapters over it.

3 From Empty Registry to READY via `.gts`

```
from pathlib import Path

from ComplexGitSync import GitRepo, GitTree, RefKind, TreeLifecycleState
from ComplexGitSync.orchestre import GtsDocument
from ComplexGitSync.registry import RegistryTranslator
from ComplexGitSync.git_tree import WorkingGitTree

# 1) Empty registry lifecycle
empty_registry = WorkingGitTree()
assert empty_registry.recompute_tree_state() == TreeLifecycleState.
   UNLOADED
```

```python
# 2) Load the CGSil1 runtime snapshot (.gts) and rebuild the dependency registry
cgshome = Path("../..")
gts_doc = GtsDocument.from_toml(cgshome / ".cgitsync/state/CGSil1.gts")
registry = RegistryTranslator.from_gts_document(gts_doc)
assert registry.lifecycle_state == TreeLifecycleState.READY

# 3) Rebuild GitTree/GitRepo identity objects from discovered registry entries
tree = GitTree()
for entry in registry.values():
    tree.add_repo(
        GitRepo(
            project_owner_name=entry.project_owner_name or "owner",
            project_name=entry.project_name or entry.name,
            gitprovider=entry.gitprovider,
            access_protocol=entry.access_protocol,
            commit_sha=entry.commit_sha,
        )
    )
```

4 Object-Level Tag Propagation

```python
# Set the same tag target across all registry entries (parent -> leaves)
tree.propagate_tag(registry, "v1.2.3")
for entry in registry.values():
    assert entry.target_ref_kind == RefKind.TAG
    assert entry.target_ref_name == "v1.2.3"
```

This object-level flow is the same state model used by the CLI and `ComplexGitSyncClient`, but exposed directly for advanced automation and tests.

5 The Operating Space: one name, one id, one Merkle root

```python
from ComplexGitSync.operating_space import OperatingSpace

registry.project_name              # the project's one name; read-only
space = OperatingSpace(registry)
space.os_id                        # SHA-256 of the name, Unicode NFC, case kept
space.gittree_root                 # H_GITTREE, computed from the tree itself
space.commit(client.git_runner, "message")   # every git operation on a READY tree
```

The tree is the reference. A `.gts` written from it records the same Merkle root, or is not written, and every document it produces names the same project. The name is set once, when the tree is built from a `.cgs` or a `.gts`; assigning it raises. A git operation on a tree that is not READY raises `TreeNotReadyError`. Clone and pull, which make a tree READY, stay on the tree (`GitTree.git`). The client methods (`commit`, `push`, ...) are unchanged and go through `OperatingSpace`.

6 READY Methods in the Client API

The same lifecycle model is exposed by `ComplexGitSyncClient`. In practice, the workflow is:

- `load(...)` is the step-1 name for direct source loading.
- `expand(...)` is the canonical step-2 name; it runs nested `.cgs` discovery from parents to leaves (recursive).
- `validate(...)` is the canonical step-3 name; it accepts both `.cgs` and `.gts` sources.
- `initialise(...)` bootstraps a tree from a `.cgs` and must finish in `READY`; a `.gts` raises `InstallFrontierError` naming `bootstrap` and `release load`. `bootstrap(...)` is the standalone entry point that clones the root too, from a `.cgs` or a `.gts`; its tree is a twin, whose memory is filed on `ComplexGitSync_<project>` (`WorkingGitTree.memory_twin`). A developer install that left a private repository on its fallback branch for want of its own branch lists it in `missing_private_branches`, each with the command that creates the branch. `configure(...)` raises `ConfigValidationError` for a project name holding `_`, and `branch(...)` raises `GitSyncError` for a branch name holding one. `pull(...)` resynchronises an existing tree.
- `git(tree, command, *args)` is the unified step-5 interface for `"commit"`, `"push"`, and `"tag"` operations. Each follows leaf-first ordering.
- `freeze_release(commit_message, *, force_tag=None, output_gts=None, stage_all=True, force_access_protocol=None)` is the one release procedure, `release freeze` on the command line. It tags `<project-name>-<suffix>`: `force_tag`, else the version in the root's `pixi.toml`, else the next number. The root's tag is annotated: its subject carries the freeze message, and its body is the project-scope State as TOML. `next_release_tag(force_tag=None)` returns that tag, the version and where the suffix came from, without changing anything. `freeze(...)` and `freeze_state(...)` are internal steps that emit the next persisted `.gts` state and have no CLI command of their own.
- `list_releases()` returns one dict per release of the project, newest first: `tag`, `version`, `recorded_at`, `by`, `state`, `made_with`, `source` (`ledger`, `tag`, `ledger+tag` or `tag only`) and `loadable`. It reads the ledger and the root repository's tags, so a release another user made is listed too.
- `load_release(name, *, workspace=None, cgs_path=None)` puts every repository back at the release's recorded commit, detached, cloning any the tree lacks, and refuses a tree with uncommitted changes; afterwards `last_release_load` (a `ReleaseLoad`: the tag, the branch the tree was on, the repositories cloned, the ones left alone) says what it did. With `workspace` it builds a new workspace from the release's State, as `bootstrap` does. `name` is the whole tag or what follows the project's name, or, when no release is called that, a State's full hash: the State is read from `.cgitsync/state/` or from the memory and loaded only when its content still hashes to that name, else `GitSyncError`. Each row of `list_releases()` also carries the freeze `message`.

```
from pathlib import Path

from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
cgspath = Path("../..")
cgshome = cgspath / "CGSil1"
```

```
# 1. load the CGSil1 test-case .cgs -> .gts LOADED
client.load("../CGSil1.cgs")

# 2. expand(.cgs/.gts) -> .gts PENDING
print(client.expand("../CGSil1.cgs"))

# 3. validate(.cgs/.gts) -> .gts READY
client.validate("../CGSil1.cgs")

# 4. clone(.cgs/.gts)
# Same default as the CLI: output_path defaults to CGSPATH=../..
client.initialise("../CGSil1.cgs")

# Equivalent explicit form:
client.initialise("../CGSil1.cgs", output_path=cgspath)

# initialise refuses, changing nothing, when a destination holds work
    that
# exists nowhere else; there is no clean-up call. Commit and push it, or
     move
# the directory aside yourself.

# 5. READY methods - unified git interface
registry = client.get_dependency_registry()
# Pull uses ROOT -> PARENT -> LEAF; every repository in the tree is a
    plain
# independent clone (there are no gitlinks) and receives its own git
    pull.
# A .gts source must be an actual snapshot path (find the current one
    via
# RuntimeStateStore.latest_snapshot_for(), the same lookup the CLI uses
    when
# a command's snapshot argument is omitted) -- unlike a .cgs source,
    there is
# no fixed, predictable filename to hard-code here.
client.pull("../CGSil1.cgs")
# pull() fetches every branch of each remote, so checkout() can join a
    branch
# somebody else pushed: it starts the local branch from the remote-
    tracking
# ref when one exists, and only otherwise from the current HEAD.
client.checkout("feature/my-branch")
# Mutations use LEAF -> PARENT -> ROOT.
client.add()
client.git(registry, "commit", "feat: update CGSil1 CGS#1")
client.git(registry, "push")
client.git(registry, "tag", "v1.2.3")

# 6. release: add, commit, pull, push, tag CGSil1-<version or number>,
    record the State
client.freeze_release("release 2026.05")
client.freeze_release("a beta", force_tag="beta")  # tagged CGSil1-beta

# Every release of the project, whoever made it, and back to one later:
for release in client.list_releases():
    print(release["tag"], release["source"])
client.load_release("beta")
```

```
# A repository the load clones takes the declared protocol (SSH by
    default);
# force_access_protocol, as in initialise() and bootstrap(), overrides
    it,
# in place or into a new workspace:
client.load_release("beta", workspace="beta-look", force_access_protocol
    ="https")
```

Every one of these that reaches a remote folds the memory first. When this tree mounts and has adopted a memory (`.cgitsync/.memory`), `push()`, `tag()`, and `freeze_release()` each call `memory_push` on it before doing their own work — regardless of `private`, since the memory is this project's own record of itself, not a configuration repository scoped by that flag. `client.last_memory_fold` holds what the most recent one of these folded: the `memory_push` result dict, or `None` when no memory is mounted or the fold could not proceed. A fold that fails — the memory declared but not yet adopted, or unreachable for any other reason — is never raised: it is turned into a `warnings.warn(...)` call instead (catchable with `pytest.warns`/`warnings.catch_warnings`, the same as `checkout`'s own build-change warning), so a Python caller sees it exactly as a CLI user would, and the write it was folding for still completes. `client.memory_declared()` answers whether a memory is mounted at all, with no side effect, for a caller that wants to know in advance.

`list_branches(private=False)`. Returns one `RepoBranches` per repository, parent-first: its `name`, the `current` branch (`None` when detached) and the sorted local `branches`. Read-only; a repository not yet cloned is left out. `private=True` narrows it to the writable configuration repositories, as for `branch()`.

```
for repo in client.list_branches():
    print(repo.name, repo.current, repo.branches)
```

`project_branches(private=False)`. Returns one `ProjectBranch` per branch of the project — the root's local and origin branches, a `closed/` branch under its original name with `closed` set. Each carries `local`, `on_origin`, `current`, `following` (repositories holding the branch they would be on under it), `missing` and `uncloned`. Reads refs only: no fetch, no State written. `private` narrows the coverage, not the branches listed.

`fetch(private=False)`. Runs `git fetch -prune origin` in every cloned repository in scope and returns one `RepoOutcome` per repository (also on `client.last_write_outcomes`): `acted` when it fetched, otherwise `detail` says why not. Moves no branch and writes no State.

`tree_branch_label()`. Returns the branch the project is on, as `status` prints `cgitsync_branch`: a branch name, `detached` or `unknown`.

`close_branch(branch_name, private=False)`. Closes a project branch across the full tree, leaf-first, locally and on the remote, never deleting anything. Like `delete_branch`, it raises `GitSyncError` in a user's tree, one that holds no private repository, before touching any repository. Each repository closes the branch it follows under `branch_name` (`<project>_<branch>` in a private/local repository; a private/distant one is skipped). A branch that exists only on the remote is closed there. Requires `READY`. Raises `GitSyncError` before touching any repository when `branch_name` is the project's own default branch, or when any repository in scope is currently on the branch it would close — the error names which. `client.last_write_outcomes` holds one `RepoOutcome` per repository visited: `acted` is `False` for a repository with no such branch or a private/distant one, never an error. `private=True` selects the writable configuration repositories instead of the project's own, the same as `branch()` (create).

```
client.close_branch("memory-dev")
for outcome in client.last_write_outcomes:
    print(outcome.name, outcome.acted, outcome.detail)
```

Before it renames anything, `close_branch` keeps every commit the branch alone holds on the project's `ancestors` branch (`<project>_ancestors` in a private/local repository) with a merge that only adds a commit, records each move as a `Relocation` in the ledger entry it writes, and refuses unless the ledger still verifies. The closed branch can then be deleted by any tool without losing a commit. `client.last_ancestry` holds what each repository's copy of the branch held, and `client.last_kept_outcomes` what keeping it did.

`branch_ancestry(branch_name, private=False)` answers, without writing anything but remote-tracking refs, what deleting a branch would lose: one `RepoAncestry` per repository, leaf-first, whose `verdict` is `"safe"`, `"recorded"`, `"needs ancestor"`, `"absent"` or `"skipped"`, with the commits only it reaches (`exclusive`) and the memory files only it holds (`content`). `delete_branch(branch_name, private=False)` deletes a *closed* branch on origin and locally, leaf-first, once everything it alone holds is kept and recorded; a move the close already recorded is reused, never written twice, and any failing step raises `GitSyncError` with nothing deleted. `preserved_branches()` names every project branch the ledger records as kept on `ancestors`, deleted ones included.

```
for answer in client.branch_ancestry("closed/feature-x"):
    print(answer.name, answer.verdict, answer.detail)
client.delete_branch("feature-x")
print(client.preserved_branches())
```

`memory_as_of(cgshome, moment, branch=None)` and `memory_list(cgshome, branch=None)` read another chapter of the memory from Git when `branch` is given — its branch, its closed name, or the copy `ancestors` keeps once it is deleted — and say which in `read_from`. `memory_explore` does the same for a chapter not checked out here, and `memory_show` finds a State that is not on disk in any of them.

7 Landing a Branch: `merge` and `refresh_private`

`merge(project_branch, private=False, ff_only=False, no_ff=False)` merges a branch across the tree, leaf-first, and returns one `(repo_name, merged_ref)` pair per repository a merge moved.

The argument is always the **project's** branch name. Each repository resolves what that means for itself, so a private/local configuration repository merges `<its default_branch>_<project_branch>` rather than a branch of the project's name, which it does not have. `merge_plan(project_branch, private=False)` returns one `(repo_name, source_ref, status, conflicting_paths)` row per repository without merging anything — what the CLI's `-dry-run` prints. `status` is `"merge"`, `"already-on-it"`, `"no-branch"` or `"conflicts"`, and the last one carries the files that block it.

Every repository in scope is checked before any is merged, so a conflict anywhere leaves the whole tree untouched. The check names the conflicting files, and counts binary files: a tracked PDF changed on both sides conflicts just as a source file does.

`merge_resolve(project_branch, private=False, ff_only=False, no_ff=False)` is the opt-in alternative. It merges one repository at a time, stops at the first that conflicts, and returns a `ResolveOutcome` naming what it merged, where it stopped, the files that stopped it, and the repositories it never reached. It gives up the all-or-nothing guarantee on purpose: that is the only way to reach the conflicted worktree a merge tool needs. `ResolveOutcome.stopped_status` is `"conflicts"`, `"unrelated"` (no common commit; nothing was attempted) or `None` when the run did not stop; `stopped_source` names the branch the stopped repository was merging, and `binary_paths` the conflicting files that are not text. `merge_resolve_all(...)` repeats it: each text conflict the merge tool resolves is committed and added to `merged`, which gathers every pass, and it stops on anything else, with `hand_command` set when no merge tool could be opened. It always ends.

`open_merge_tool(repo_name)` then opens one repository's conflicts in a merge tool. It returns `None` once the tool has run, or the command to run by hand when no tool is available — a missing editor is a normal outcome, not an error. A `merge.tool` you configured always wins, and your Git configuration is never written.

`refresh_private()` is the other direction, and what `pull -private` runs: it fetches and merges each private/local repository's base branch into the derived branch it is on, so a settings branch does not drift behind the project's while a feature branch is open.

```
client.merge("multi-branch")                          # the project's own repos
client.merge("multi-branch", private=True)     # ComplexGitSync_multi-
    branch
                                                      #   -> ComplexGitSync
client.merge("multi-branch", all_writable=True)   # both halves, one
    pass

# Merging into a branch that is not checked out, in one call.
client.merge_into_plan("multi-branch", "main")   # what it would do, per
     repo
client.merge_into("multi-branch", "main")          # checkout and merge,
    together
client.build_installed_from("main")                # the version that
    branch carries
client.refresh_private()                              # the other way round
```

`all_writable`: the API half of `-all`. `add`, `commit`, `push`, `merge`, `merge_resolve` and `merge_plan` each take `all_writable=False` alongside `private=False`. Setting it runs the call across this project's own repositories *and* its writable configuration ones in a single pass. The two are mutually exclusive and passing both raises; leaving both unset keeps today's behaviour exactly. Read-only configuration repositories are never written to under any combination — the parameter maps to `RepoScope.WRITABLE`, which is narrower than `RepoScope.ALL`.

8 Targeting a Single Path: `add` and `remove`

`add(paths=None)` and `remove(paths)` accept one or more filesystem paths (each relative to `CGSHOME`, or absolute) instead of operating on the whole tree:

```
# Stage only these two files, each in the repo that owns it -- every
# other repo in the tree is untouched.
client.add(paths=["notes.md", "deps/leaf/src/main.py"])

# Remove a single tracked file: deleted from disk, removal staged.
client.remove(["deps/leaf/obsolete.txt"])
```

Both resolve each path to its owning repo through the same helper, `resolve_repo_for_path(tree, path)` (`git_tree.py`) — the deepest (most specific) repo whose `absolute_path` contains the resolved path wins, so a nested child is correctly preferred over its parent. A path outside every repo in the tree raises `GitSyncError` immediately, before anything is staged or removed; for `remove`, a path that resolves to a directory or does not exist raises the same way rather than partially applying. `add()` with no `paths` keeps today's exact behaviour: every repo staged in full, leaf-first.

`remove` is a plain `git rm` (deletes from disk, stages the removal) — unrelated to `GitRunner.rm_cached` (index-only, used internally by `import_submodules` to drop a gitlink while keeping the child's working tree); the two are not interchangeable.

9 `.gitignore` Lifecycle Sync and Commit Identity

`initialise_cgs`, `initialise_cgs_document`, `restart`, and `pull` (for `.cgs` sources) all accept the same three optional keyword arguments, mirroring the CLI's `-commit-gitignore`, `-git-user-name`, and `-git-user-email` flags:

```
from ComplexGitSync import ComplexGitSyncClient, MasterConfig

client = ComplexGitSyncClient()
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    commit_gitignore=True,        # stage/commit/push each changed .
        gitignore
    git_user_name="cgitsync-bot",
    git_user_email="bot@example.com",
)

# The identity above is persisted to CGSHOME/.cgitsync/master.toml, so a
# later call on the same workspace picks it up without repeating it:
client.initialise_cgs("../CGSil1.cgs", output_path=cgspath,
   commit_gitignore=True)
assert MasterConfig.resolve_identity(cgshome, client.git_runner) == (
    "cgitsync-bot",
    "bot@example.com",
)
```

All three keyword arguments default to `False`/`None`: by default, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled and has its `.gitignore` updated, but nothing is staged, committed, or pushed, and the commit identity — were a commit ever made — would be whatever `git config user.name`/`user.email` already resolves to locally. `MasterConfig` (`master.py`) is the class behind the identity override: `configure()` sets it in memory for the current process, `persist(cgshome, ...)` writes it to `CGSHOME/.cgitsync/master.toml` (and also updates the in-memory value), `load(cgshome)` reads a previously persisted override back in, and `resolve_identity(repo_path, git_runner)` returns the `(user_name, user_email)` pair — or `(None, None)` to defer entirely to local git config. This file is workspace-local configuration, not part of the `.cgs`/`.gts` project spec, and is preserved never touched by `initialise`.

10 Forcing a Clone Protocol

`initialise_cgs`, `initialise_cgs_document`, `bootstrap`, and `clone_cgs` additionally accept `force_access_protocol`, mirroring the CLI's `-force-protocol` flag on `initialise`, and `bootstrap` (`restart`/`pull` do not accept it):

```
# Every repo this run clones -- root siblings and any child discovered
# later from a nested .cgs in a different repo -- is cloned over https,
# regardless of what its own .cgs entry says. No .cgs file is read
# differently or written.
```

```
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    force_access_protocol="https",
)
```

`"ssh"` or `"https"`; `None` (the default) leaves every entry's own `.cgs`-declared `access_protocol` untouched. The override lives only in memory for the duration of the call – it is applied at the single point every clone's remote URL is built (`ComplexGitSyncClient._build_remote_url`), so it reaches root siblings and nested-discovered children alike without `cgs_format.py` parsing, or any `.cgs` file on disk, ever being touched differently.

11 Repository Discovery: `discover_repos`

`discover_repos(root_dir=None, *, max_depth=5, output=None)` scans a directory for git repositories already checked out on disk and drafts a `.cgs` from what it finds — the entry point for adopting a project that exists but has no specification yet:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Read-only scan - nothing is cloned, fetched, or modified
report = client.discover_repos("/path/to/checkout")
for repo in report.repos:
    print(repo.relative_path, repo.identifier, repo.branch)

for warning in report.warnings:
    print("unresolved:", warning)

# Write the draft out: a relative output lands inside the scanned
# directory, so this writes /path/to/checkout/draft.cgs
report = client.discover_repos("/path/to/checkout", output="draft.cgs")
print(report.written_to)
```

The method returns a `DiscoverReport` dataclass with fields `root` (the scanned directory), `repos` (a tuple of `DiscoveredRepo`), `cgs_entries` (authoring-form repo tables ready for `configure()`), `warnings`, `project_name`, and `written_to` (the path the draft was written to, or `None`). Each `DiscoveredRepo` carries `relative_path`, `absolute_path`, `remote_url`, `identifier`, `branch`, and `has_cgs`.

Repository addresses are resolved through the existing `parse_repo_id()` grammar; a repository with no `origin` or an unrecognised provider is reported in `warnings` and omitted from `cgs_entries` rather than guessed at. Passing `output` raises `GitSyncError` when nothing resolvable was found, so an empty draft is never written.

12 Submodule Migration: `import_submodules`

`import_submodules(repo_root, *, apply=False, recursive=False)` converts git submodules in a local repository to plain ComplexGitSync nested clones. It is what the CLI's `submodules report` (`apply=False`) and `submodules import` (`apply=True`) run:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

```

```
# Dry run - inspect without changing anything
report = client.import_submodules("/path/to/repo")
for sub in report.submodules:
    print(sub.name, sub.path, sub.url)

# Apply - convert every level, root first
report = client.import_submodules(
    "/path/to/repo",
    apply=True,
    recursive=True,
)
print(report.converted)   # tuple of converted submodule names
```

The method returns an `ImportSubmodulesReport` dataclass with fields `submodules` (all entries found), `applied` (whether the conversion ran), `converted` (names of converted submodules), and `scan_root` (the repository the call was pointed at, which every reported path can be expressed from). When `apply=False` (the default), no files are modified. When `apply=True`, each submodule is removed from the index (`git rm -cached`), its stanza is removed from `.gitmodules`, and the parent's `.gitignore` is updated to ignore the child path. The staged changes should then be reviewed and committed manually.

`recursive=True` also converts any submodule that has its own checked-out `.gitmodules`, at any depth. That walk follows the submodule graph declared by `repo_root`'s own `.gitmodules`, so a repository whose holder has already been converted is no longer reachable — which is why the conversion of a whole tree happens once, and last (see `init_from_submodules` below).

13 Adopting a Submodule Tree: `init_from_submodules`

`init_from_submodules(repo_root, *, cgs_path=None, max_depth=5, dry_run=False, force=False, force_access_protocol=None)` performs the whole adoption of a submodule-based checkout, in the one order that works. It is what the CLI's `submodules init` runs:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Report the plan without writing, cloning, or converting anything
plan = client.init_from_submodules("/work/project", dry_run=True)

# Adopt: discover -> write .cgs -> initialise -> convert every submodule
report = client.init_from_submodules("/work/project")
print(report.cgs_path)                    # the .cgs used
print(report.tree.is_ready())             # True
print(report.import_report.converted)     # converted submodule names
```

Each step delegates to the method that already owns it (`discover_repos`, `initialise_cgs`, `import_submodules`); the value this method adds is the ordering and the preconditions. `initialise_cgs` adopts the root in place but deletes and re-clones every other repository from its remote, where the submodules are still declared — so converting first is undone for every non-root repository, and a repair pass cannot reach any deeper level once the root's `.gitmodules` is gone.

The method returns an `InitFromSubmodulesReport` with fields `root` (which is also `CGSHOME`), `discover` (the `DiscoverReport` from step 1), `cgs_path`, `cgs_written`, `import_report`, `tree`, and `dry_run`. Nothing is committed: the conversion touches every repository that held a submodule, so `branch`, `checkout`, `add`, and `commit` stay explicit, separate calls. A `GitSyncError` is raised before anything is written when the discovery resolved nothing, when the project name does not match the directory name (`CGSHOME` is `<parent>/<project-name>`), or when `repo_root` has no `.gitmodules` to convert (`force=True` overrides the last one).

14 Looking at a memory: `memory_status`, `memory_list`, `memory_show`, `memory_explore`

The four read-only questions the CLI's `memory` group asks, each returning plain data rather than text:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

status = client.memory_status("/path/to/CGSHOME")
status["states"], status["entries"]      # how much is remembered
status["pending_entries"]               # of those entries, how many are
    not yet folded into the memory repository
status["verification"]  # verified / no-history / legacy / corrupt /
   time-inconsistent / incomplete
status["latest_toolchain"]["git"]        # what produced the newest
   record

for row in client.memory_list("/path/to/CGSHOME"):
    row["state"], row["recorded_at"], row["commands"]

state = client.memory_show("/path/to/CGSHOME", "2acdc98b")   # hash or
   prefix
state["repos"], state["entries"]
state["tree"]                            # this State's own topology,
   like view_tree()
state["environments"]                    # [{"id": "env(<hash>)", "path
   ": ..., "record": ...}]
for entry in state["entries"]:
    for commit in entry["commits"]:
        commit["message"], commit["repository"], commit["published"]
state["published"]                       # remote and ref each commit
   went to

answer = client.memory_as_of("/path/to/CGSHOME", "2026-09-30T17:00")   #
    UTC unless an offset
answer["entry"]                          # last entry recorded at or
   before it, or None
answer["entry"]["seq"], answer["entry"]["state"]   # the hash
   memory_show takes
answer["reliable"], answer["history"]    # False + "time-inconsistent"
   when a clock ran backwards
answer["findings"]                       # why, as ‘verify‘ would say

environment = client.memory_show_environment("/path/to/CGSHOME", "env(<
   hash>)")
environment["record"]["machine"], environment["record"]["tools"]

answer = client.memory_explore("/path/to/CGSHOME")
answer["branch"]                         # the memory branch checked out
    here, or None
for commit in answer["commits"]:         # published commits, newest
   push first
```

```
    commit["repository"], commit["branch"], commit["sha"], commit["
        message"]

timeline = client.memory_explore("/path/to/CGSHOME", timeline=True)
for entry in timeline["entries"]:         # every ledger entry, in order
    entry["seq"], entry["command"], entry["state"], entry["commits"],
        entry["published"]
    # entry["state"] is what memory_show(cgshome, entry["state"]) takes

report = client.memory_verify_published("/path/to/CGSHOME")   # or state
   ="<hash>"
report["moved"]                           # how many pushes the remote no
    longer contains
for row in report["rows"]:                # one per push recorded with
   the remote's answer
    row["verdict"]                        # "attested", "moved" or "
        unreachable"
    row["repository"], row["ref"], row["sha"], row["remote_commit"], row
        ["observed_at"]
```

Commit messages come back whole. Shortening a long one to a single line is the caller's decision, which is why the CLI has `-full` and this method does not: a caller reading the memory from Python wants the message that was written, not the one that fitted.

`memory_show` raises `GitSyncError` when the prefix matches no State, or more than one.

`memory_explore`'s default view answers what a colleague pulling this branch would see — commits with no publication row are left out, because that is `memory_show`'s question, not this one. `branch=` asks for a memory branch other than the one checked out on this disk; today only the checked-out one can be read this way, and a mismatch raises `GitSyncError` naming `memory_clone(branch=...)` rather than answering for the wrong branch.

The commands that give a memory a repository of its own have the same mirror. `add_memory_repo_cgs` is the one the owner named; the CLI spells it `memory mount`.

```
client.repo_create("github:you/.memory", private=True)
#   {"created": "created" | "exists" | "unavailable", "command": "gh
   repo create ..."}

client.add_memory_repo_cgs("project.cgs")         # one entry, comments
   kept
client.memory_adopt("/path/to/CGSHOME")           # .cgitsync/.memory,
   fresh and empty
client.memory_push("/path/to/CGSHOME")            # folds .cgitsync's
   pending content in first
client.memory_branch("/path/to/CGSHOME", "main") # the branch a merge
   will need

# Only for a memory mounted before this command existed, directly at
# .cgitsync instead of .cgitsync/.memory:
client.memory_migrate("/path/to/CGSHOME", "project.cgs")
```

`repo_create` never raises for a repository that already exists — that is `"exists"`, an ordinary answer — and never for a provider tool that is absent or signed out, which is `"unavailable"` carrying the command to run. It raises only for a provider no tool is known for, and for a tool that failed for some other reason. `memory_list` includes a State that entries name but that is not on disk, with `path` set to `None` — `verify` reports it as `MISSING_STATE`, and this is how a caller finds out which one.

14.1 Starting a memory's history over: `memory_reboot`

```
answer = client.memory_reboot("/path/to/CGSHOME")
answer["folded"]         # pending records folded in before archiving
answer["archived_from"]  # the branch's name before the reboot
answer["archived_to"]    # "<branch>.archived-<YYYYMMDD>", real, on
    origin
answer["exported"]       # .cgitsync/.memory/.cgs/<project>-v<N>.cgs
answer["branch"]         # the original name, fresh: .cgs/ kept, else
                          # cleared, one State committed and pushed

# A fresh start from the very first adopt, instead of inheriting
    whatever
# the fallback branch already holds:
client.memory_adopt("/path/to/CGSHOME", remote=remote, branch=branch,
    reboot=True)
```

Nothing is ever force-pushed or deleted: the old branch is pushed to origin under its archived name *before* the old name is removed from origin, so the commits it holds are reachable under some name on the remote for the whole operation, never only locally. The fresh branch's own genesis commit is pushed too, immediately, rather than left for the next `memory_push` — the old name is already gone from origin by then, so a bootstrap on another machine in between would otherwise find no branch under this project's name at all. `memory_reboot` raises `GitSyncError` when the mount is not a repository yet, and when today's archived name already exists — a second reboot the same day needs a name of its own rather than colliding with the first.

14.2 Keeping a branch's memory when its project branch is merged: `memory_merge`

```
answer = client.memory_merge("/path/to/CGSHOME", "feature-x", into="main
    ", keep="ours")
answer["status"]   # "merge" | "fast-forward" | "already-merged"
answer["detail"]   # "kept the memory of MyProject; MyProject_feature-x
    is kept as history"
answer["source"], answer["target"]    # the memory branches, derived from
     the project branches
answer["commit"]   # the merge commit, or None when nothing was due
answer["pushed"]   # False for a defaulted memory or one with no remote
```

`keep` is `"ours"` (keep `into`'s memory) or `"theirs"` (keep the source's), Git's meaning, and is required: any other value raises `GitSyncError` before anything is touched. `into` defaults to the project branch checked out. A memory is never merged file by file; the commit takes one side's files whole and records the other as its second parent. A missing source or target memory branch raises `GitSyncError` naming it (and, for the target, `memory_branch`); a push the remote would reject raises one that says the merge is already done locally. Nothing is forced. `memory_branch(cgshome, project_branch, *, push=True)` raises `GitSyncError` when the branch already exists, here or on the remote, and otherwise returns `created`, `set_up` (true when it also wrote `config-memory.cgs` and adopted `.self-history`) and `pushed`.

14.3 Recording one piece of agent work: `self_history_add`

```
from ComplexGitSync.memory.self_history import AgentInfo,
    ConformityCriterion, ConformityScore

worker = AgentInfo(role="Dev", vendor="vendor-name", model="model-name")
orchestrator = AgentInfo(role="Orchestration", vendor="vendor-name",
    model="model-name")
```

```
conformity = ConformityScore(
    spec_respect=ConformityCriterion(score=33, basis="measured",
       reasoning="lint and test both pass"),
    gating=ConformityCriterion(score=33, basis="measured", reasoning="
       nothing private pushed"),
    quality=ConformityCriterion(score=30, basis="asserted", reasoning="a
        reasonable first pass"),
)

path = client.self_history_add(
    "/path/to/CGSHOME",
    ticket="AgentReport",
    goal="Implement WP1: the record format and the add command.",
    action="Wrote memory/self_history.py and self_history_add.",
    worker=worker,
    orchestrator=orchestrator,
    conformity=conformity,
    lint_passed=True,
    tests_passed=True,
)
```

Writes one record to `.cgitsync/.self-history/<hash>.toml`, content-addressed the same way a State or an Environment record is. `contract` (the current signed `AgentContractRecord`'s own hash) and `checks.status_errors` (this workspace's `errors=` count) are filled in from what the client can itself observe; every other field, including `lint_passed`/`tests_passed`, is the caller's own account — `self_history_add` cannot run `pixi run lint` or `pixi run test` itself, since `subprocess` stays confined to `git_runner.py`, and that module runs Git, not Pixi.

`AgentInfo.role` is validated against `.localSpec/AGENT.md`'s six-role roster; `ConformityCriterion.basis` against `"measured"`/`"asserted"` only. Both raise `ValueError` on anything else, the same way `goal`/ `action` raise for more than three lines.

`ConformityScore` is out of 100: `spec_respect` 0–33, `gating` 0–33, `quality` 0–34, the total their plain sum. A score outside its range raises `ValueError`. `ConformityScale` (in `memory.conformity_scale`) holds those maxima and renders a score with them. The optional `ConformityScore.explanation` is the orchestrator's short account of how the total came about; it is left out of the stored record when empty.

`state_before`/`state_after` are **verified against the ledger**, not merely shape-checked — deliberately a ledger question, not a `.cgs`/discovery one: a `.gts` is a static snapshot, so whether a citation names something real is answered the same way `verify` already answers it, via `MISSING_STATE`/ `STATE_DIGEST_MISMATCH` — never by re-parsing or re-deriving something from a transformed document. Concretely, each citation, if given, must name a State some ledger entry actually recorded, on disk, still hashing to its own name; one that fails any of the three raises `GitSyncError` rather than being recorded as though it were fact. `state_after` is additionally **observed**, not merely validated, when omitted: the ledger's own most recent entry at the moment `self_history_add` runs.

`memory_push` folds this pending directory into `.cgitsync/.memory/.self-history/` and pushes it — *before* folding and pushing `.memory` itself, once self-history has been adopted (below). A tree that never adopted it sees no change: the fold is a no-op the moment `.self-history`'s own mount has no `.git`.

14.4 Adopting self-history: `self_history_adopt`

```
result = client.self_history_adopt("/path/to/CGSHOME", owner="you")
result["mount"]    # .cgitsync/.memory/.self-history
result["branch"]   # .memory's own branch, unless a different one was
   given
```

```
result["remote"]  # the SSH URL self-history was adopted from
```

`self_history_adopt` is the *one place* this fact is ever decided: it writes `config-memory.cgs` the first time, for a brand-new project or as the explicit retrofit for a `.memory` adopted *before* self-history existed (this project's own, among others). Every precondition here raises `GitSyncError` instead of returning quietly — `.memory` not yet adopted, self-history already adopted, no owner resolvable, or the remote not reachable.

There is no separate opt-in flag beyond that one decision: `memory_adopt` follows it automatically and silently afterward, whenever `.memory`'s own already-fetched content already carries `config-memory.cgs` — typically because a project's own memory branch inherited it from a fallback base branch that already had it, the same mechanism that already lets a new project's memory share history with another. This is deliberately *not* a check against `self.registry`: a `.gts` is a static snapshot of an already-discovered tree, and `nested_config` (the `.cgs`-side declaration that makes `.self-history` a discoverable child of `.memory`) is `.cgs`-only information a `.gts`-loaded registry never carries in the first place — asking the registry would be the wrong layer, not a gap a runtime probe should paper over. Reachability is checked too, but only *after* that question, to decide whether an already-declared repository can actually be fetched right now.

`memory_clone` brings self-history back too, reading which repository to clone from `config-memory.cgs` itself rather than re-deriving it, the moment that file says self-history was adopted upstream. `memory_reboot` leaves `.self-history` entirely alone by construction: it only ever touches `.memory`'s own mount.

14.5 Reading self-history: `memory_self_history`

```
records = client.memory_self_history("/path/to/CGSHOME")
records[0]["ticket"]         # the short ticket name
records[0]["conformity"]     # {"spec_respect": {...}, "gating": {...}, "
    quality": {...}}
```

Every record this workspace holds, folded and pending, oldest first — what the CLI's `self-history list` prints — each as the same dict `SelfHistoryRecord.to_dict()` produces – consultation only. An empty list means no self-history has been written here yet, not an error.

15 Machine-readable output: `status_json` and `verify_json`

Two methods return the same answers as `status()` and `verify()`, serialised as one JSON object each. They are what the CLI's `-json` flag prints, so a Python caller and a shell pipeline read exactly the same fields.

```
import json
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/snapshot.gts")

payload = json.loads(client.status_json())
payload["cgitsync_branch"]           # the branch the tree is on
payload["profile"]                   # "user" or "dev" (client.registry.
    profile)
payload["os_id"]                     # OperatingSpace(client.registry).
    os_id, or None
payload["summary"]["repos"]          # how many repositories it holds
payload["repositories"][0]["sync"]   # per repository, as the table shows
     it
```

```
chain = json.loads(client.verify_json("/path/to/CGSHOME"))
chain["status"]                          # "clean" or "findings"
```

`verify_json` keeps the report it built on `client.last_verify_report`, so a caller needing both the object and the verdict does not verify the chain twice — which under `repair=True` would mean repairing twice.

The shape of both objects lives in `json_render.py`, defined once for every command rather than per command, and carries a `schema_version`. The promise attached to it is **additive only**: new fields may appear; existing fields do not change meaning and do not vanish.

16 Environment observation and drift

The client exposes the same read-only Environment operations as the CLI:

```
from ComplexGitSync.cgs_format import CgsDocument
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/CGSHOME/.cgitsync/state/<hash>.gts")

observed = client.environment()
print(observed.architecture, observed.pixi_platform, observed.digest())

requirements = CgsDocument.from_toml("/path/to/project.cgs")
drift = client.check_environment(requirements)
if not drift.matches:
    print("missing", drift.missing)
    print("older", drift.older)
print("observed but undeclared", drift.undeclared)
```

`environment()` returns a secret-free, content-addressable `TreeEnvironment`. Credentials are booleans only; manifests are tree-relative paths and digests, never copied contents. When a source `.cgs` is available, `check_environment()` may be called without an argument. It returns data and installs nothing.

17 A memory as a repository: `memory_init`, `memory_clone`, `memory_push`

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/CGSHOME/.cgitsync/state/<hash>.gts")

proposal = client.memory_init("/path/to/CGSHOME")
proposal["entry"]          # the .cgs entry that mounts the memory
proposal["branch"]         # this project's branch of the memory
    repository
proposal["create_with"]    # the command that creates it -- never run for
    you

client.memory_push("/path/to/CGSHOME")                        # commit +
    push
client.memory_clone("/other/workspace", owner="flipoyo")     # onto a new
     machine
```

`memory_clone` needs no loaded project — that is the case it exists for — so give it an `owner` and a `branch` when nothing is loaded, or a `remote` address directly for a memory that does not live where the convention says. It raises `GitSyncError` rather than writing over a memory already present, or when the branch it wants has never been pushed.

17.1 A developer's tree with no memory: `memory_setup`

A tree holding any private repository is a developer's (`client.registry.profile` is `TreeProfile.DEV`); one holding none is a user's. A developer's memory is meant to be synced, so when the `.cgs` declares none these three methods offer one. None of them prompts: asking is the CLI's job.

```
proposal = client.memory_setup_proposal()  # None for a user tree, or a
    declared memory
proposal["owner"]        # guessed: most frequent private owner, else
   the root's, else None
proposal["create_with"]  # "gh repo create you/.memory --private"
proposal["warning"]      # the plain-words warning, naming the fix

result = client.memory_setup(provider="github", owner="you", name=".
   memory")
result["done"]           # ["create", "declare", "adopt"] on success
result["failed"]         # None, or the step that stopped it

client.memory_setup_decline()  # asked once: remembered in .cgitsync/
```

`memory_setup` runs `repo_create`, `add_memory_repo_cgs` and `memory_adopt` in that order and stops at the first that fails; it raises `GitSyncError` only when there is nothing to set up or no owner is known. A command that records a State in such a tree still records it, sets `client.memory_setup_due`, and emits a `MemorySetupWarning` (`orchestre.memory_setup`) — a warning, never a refusal.

Calling `memory_push` by hand, as above, is for settling the memory on its own. *READY Methods in the Client API*, earlier in this chapter, covers the more common case: `push`, `tag`, and `freeze_release` each call it automatically, first, once a memory is mounted and adopted.

18 The shared register: `omniscience_mount`, `omniscience_record`, `omniscience_register` and `omniscience_observe`

```
mounted = client.omniscience_mount(None, "/path/to/CGSHOME", contribute=
   False)
mounted["added"], mounted["cloned"], mounted["branch"]   # declared in
   the .cgs, cloned at .omniscience

result = client.omniscience_record("/path/to/CGSHOME", "3f9a")  # a
   State's hash or prefix
result["record"]            # the record as a TOML-ready table
result["record_hash"]       # its hash: the name of register/<hash>.toml
result["os_state_hash"]     # H_OS_STATE
result["leaf"]              # its leaf in the global tree, as hex

done = client.omniscience_register("/path/to/CGSHOME")      # default:
   the memory's newest State
done["record_hash"], done["appended"], done["retries"]       # appended
   is False when it was already there
done["checkpoint"]["status"]   # extended / unchanged / rewritten /
   refused
```

```
done["checkpoint"]["unaccepted"]  # [{"branch", "path", "reason"}]:
   proposed files that did not validate

seen = client.omniscience_observe("/path/to/CGSHOME", None)   # None:
   this tree's project
seen["observations"]             # one table per finding appended
seen["report"]["verdict"]        # ok | warning | failure
client.omniscience_observe(home, None, confirm_deleted="github:o/r",
   reason="owner deleted it")
```

`omniscience_mount` takes the `.cgs` to edit (`None` means the one the tree was built from) and refuses a tree that holds no private repository. `omniscience_record` reads the committed memory and raises `GitSyncError` for a State that is not in it, a memory with uncommitted changes, or a memory commit that is not on its remote. Neither writes to the register.

`omniscience_register` and `omniscience_observe` do, and only by appending: they raise `GitSyncError`, before touching anything, in a tree that holds no private repository, with the register not mounted, mounted read-only, or holding uncommitted changes. `omniscience_register` validates the record against the State on disk, the project's id and the memory's remote, commits it and pushes fast-forward only, retrying a push another contributor beat and never forcing, then extends the global checkpoint; a checkpoint whose parent is no longer an ancestor of the remote's is reported as `rewritten` and neither extended nor repaired. `omniscience_observe` probes only when called. `report["verdict"]` is `warning` while a repository is `NOT_FOUND` and `failure` once that has held for two observations at least seven days apart (`omniscience_completeness.PERMANENT_AFTER`); `confirm_deleted` records the owner's note as a signed commit and refuses a target the host still serves. The pure side is importable: `OmniscienceRecord`, `GlobalIndex` and `GlobalIntegrity` compute a record's hashes, the ordered leaves and `H_GLOBAL` without Git; `Observation` and `CompletenessReport` do the same for observations.

19 Asking the register: `omniscience_get` and its eight siblings

```
home = "/path/to/CGSHOME"        # a workspace with the register mounted
    (read-only is enough)

client.omniscience_get(home, "MyProject")                        #
   registered? os_id, branches, States
client.omniscience_branches(home, "MyProject")                   # each
   branch and its newest State
client.omniscience_history(home, "MyProject", branch="main")     # every
   registration, oldest first
client.omniscience_proof(home, "MyProject", "3f9a")              # the
   checked inclusion proof
client.omniscience_verify(home, "MyProject", "3f9a")             # the
   proof, and the register's state
client.omniscience_memory(home, "MyProject", "main")             # the
   registered .memory, read and checked
client.omniscience_state(home, "MyProject", "3f9a", destination="
   restored.gts")
client.omniscience_status(home, "MyProject", "memory")           # a
   repository: registration + newest observation
client.omniscience_report(home, "MyProject")                     # the
   register, the proofs, proposals, completeness

answer = client.omniscience_get(home, "MyProject", checkpoint="ab12")
   # as the register stood at a checkpoint
```

```
answer["status"]      # verified / invalid / unavailable / unauthorized /
     not_registered
answer["detail"]      # None, or e.g. remote_unavailable /
    commit_unreachable / invalid_proof / ledger_corrupt / not_observed
answer["reason"]      # one plain sentence; the other keys are the answer
    's data
```

Each method returns one flat table: `status`, `detail` and `reason`, then the data of that question. They only read: the register is fetched into the mount's remote-tracking refs, a registered `.memory` is cloned into a temporary directory under `.cgitsync/` and removed, and nothing is written to the register, to a memory or to the tree. Anyone the provider lets read the register may call them; they add no check of their own. A refusal by the provider is `unauthorized`, a remote that did not answer is `unavailable`, and neither is ever turned into "deleted".

`proof_verified` in an answer is the recomputed audit path against the checkpoint's root and size, and says that the registration is in the index, nothing more. `omniscience_verify` adds `register`: `state` is `intact`, `attested`, `rewritten` or `contradicted`, with `reason` and the `protections` that were checked. `omniscience_memory` and `omniscience_state` add `memory`, `states`, `ledger` and `state`; a State that no longer hashes to its name adds `diagnosis` (`located` or `unavailable`) and `located`, the repositories whose own hash fails. `omniscience_state` writes `destination` only after the read verified and raises `GitSyncError` rather than overwrite a file. The public object behind them is `Omniscience(client, cgshome)` in `ComplexGitSync.orchestre.omnis` with `get`, `branches`, `history`, `get_state`, `get_proof`, `verify`, `memory`, `repository_status` and `integrity_report`, each returning an `Answer` (`.to_dict()` is what the client returns). The pure side is importable: `Answer`, `OmniscienceStatus` and `MemoryDetail` (`omniscience_answer`) and `GlobalInclusionProof` (`inclusion_proof`).

20 One writer per workspace: the workspace lock

Every client method that writes a repository or the workspace's state area — `add`, `commit`, `push`, `tag`, `checkout`, `merge`, `pull`, `freeze_release`, `memory_push` and the others listed in `orchestre.workspace_locking.WRITING_METHODS` — holds `.cgitsync/.lock` for its whole run, so a Python caller is covered by the same code as the CLI. A call that finds the lock held by another process raises `WorkspaceBusyError` (a `GitSyncError`) before touching any repository; read-only methods never take or wait for it. A writing method that calls another (`push` calling `memory_push`) does not meet its own lock.

```
from pathlib import Path
from ComplexGitSync.workspace_lock import WorkspaceBusyError,
    WorkspaceLock

try:
    client.commit("message")
except WorkspaceBusyError as busy:
    print(busy)                    # who holds it, since when, and
        whether it is stale

lock = WorkspaceLock(Path("/path/to/CGSHOME") / ".cgitsync")
lock.holder()                      # LockHolder(pid, started_at,
    command) or None
lock.clear()                       # what \texttt{cgitsync unlock}
    does; stale locks only
lock.clear(force=True)             # --force
```

A writing command whose ledger entry could not be written emits a `UserWarning` (“this run’s State was written but its ledger entry was not”) as well as the `ledger_append_failed` run-log event.

21 Ledger Integrity: `verify`

`verify(cgshome, *, repair=False)` checks the hash-chained `.cgitsync/lgr` ledger under `cgshome` for tamper-evidence. The CLI’s `verify check` calls it with `repair=False`, and `verify repair` with `repair=True`:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
report = client.verify("/path/to/CGSHOME")

if report.is_clean:
    print("ledger is clean")
else:
    for seq, finding, detail in report.findings:
        print(seq, finding.name, detail)

# Repair a stale HEAD cache (never rewrites or deletes an entry)
client.verify("/path/to/CGSHOME", repair=True)
```

Returns an `integrity.VerificationReport`. Read `report.state` first: it is one of four `HistoryState` members — `VERIFIED`, `NO_HISTORY`, `LEGACY`, `CORRUPT` — and `report.is_verified` is `True` only for the first.

`report.is_clean` is a narrower question: it means “no findings were recorded”, which an empty ledger also satisfies. Use it to ask whether anything is *wrong*; use `state` to ask what was actually established. `findings` holds `(seq, Finding, detail)` tuples.

Every command that writes a State now appends an entry to `.cgitsync/lgr/`, so a workspace used since that landed answers `VERIFIED`. A workspace whose only history predates it answers `LEGACY`: the single-file register is still read, and nothing writes it any more. `Finding` enumerates `BROKEN_LINK`, `BAD_ENTRY_HASH`, `SEQ_GAP`, `SEQ_DUPLICATE`, and `HEAD_STALE` for what this method actually checks (chain linkage and the `HEAD` cache); its remaining three members (`MISSING_STATE`, `ORPHAN_STATE`, `STATE_DIGEST_MISMATCH`) are reserved for a future store-level verification pass that cross-references entries against the actual `state/<hash>.gts` files on disk.

With `repair=True`, a stale `HEAD` cache is corrected in place. Ledger entries are never rewritten or deleted by this method, repair or not — a broken chain is reported, not silently healed.

22 Diagnosing and Repairing a Git Situation: `autofix`

`autofix(*, error=None, repo_name=None)` reads the error the tree’s last command actually failed with, and runs whichever registered repair recognises it — the same command `cgitsync autofix` wraps (§?? of the user guide):

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load("/path/to/CGSHOME/snapshot.gts")

# error=None reads .cgitsync/logs/*.log's most recent failing command
outcome = client.autofix()

# Or hand it an error directly -- useful from a script that already
# caught one, or in a test
outcome = client.autofix(
    error="fatal: Not possible to fast-forward, aborting.",
    repo_name=".memory",
)

print(outcome.repaired, outcome.detail)
```

With no failing command in the log, `autofix()` falls back to the tip commit of each writable repository, judging only commits authored under the repository's own `user.email`. It returns an outcome with `repaired=False` whose `detail` reports each message that breaks the tree's commit-message rule or looks shell-damaged, with the commands that extract it intact, and changes nothing; `NoMatchingRepairError` is raised when no tip commit is reported.

Returns an `autofix.RepairOutcome`: `repaired` (bool) and `detail` (str, one line naming what changed or why nothing did). `repaired=False` is not a failure — it also covers the case where the situation turned out not to need this repair at all (e.g. a merge that applied cleanly on its own).

Raises `autofix.NoMatchingRepairError` (mapped to the CLI's `EXIT_REFUSED`, never a traceback, when reached through `cgitsync autofix`) when no failing command is found to read, or when nothing registered recognises the error — `autofix` refuses rather than guessing. It can also raise the ordinary `errors.GitSyncError` a repair itself raises, for example when the two sides of a divergence are not disjoint enough to reconcile safely without a person's judgement.

`autofix` is deliberately not one growing class: each situation it recognises lives in its own `ComplexGitSync.autofix.repair_*` module (today, `repair_divergent_user` — a private repository whose branch diverged because two machines wrote to it independently, where the content itself, like `.memory`'s hash-chained ledger, carries an ordering invariant a plain `merge` cannot see). Adding a new situation this method can handle means adding a new module and registering it, never editing an existing one to grow another branch.